



THE AGENTIC AI ENGINEER

The 2026 *Agentic* AI Engineer Roadmap

A complete, production-grade curriculum — from Python foundations to multi-agent systems, guardrails, and AWS deployment.

26

WEEKS

9

PHASES

60

MODULES

3

CAPSTONES

Balaji Chippada
AI Engineer & Educator

balajichippada.com/roadmap
June 15, 2026

The roadmap at a glance

Nine phases over twenty-six weeks. Each builds directly on the last — the order is the point. Difficulty is relative to where you are when you reach it, not absolute.

#	PHASE	TIMELINE	MODULES	DIFFICULTY
01	Python Foundations Python + Async Engineering	Weeks 1–3	6	
02	The Mental Model of an LLM LLM Mental Model	Week 4	5	
03	Prompt Engineering & API Access Prompt Engineering	Weeks 5–7	7	
04	RAG + Evaluation Ingestion Pipeline + RAG	Weeks 8–12	9	
05	Tools, MCP, and Single Agents Tools, MCP & Single Agents	Weeks 13–16	8	
06	Memory & Context Engineering Memory + Context Engineering	Weeks 17–19	7	
07	Multi-Agent Orchestration Multi-Agent Orchestration	Weeks 20–22	8	
08	Guardrails & LLMOps Guardrails + LLMOps	Weeks 23–24	4	
09	Cloud Infrastructure & Deployment Cloud + Deployment	Weeks 25–26	6	

01 Python Foundations

Every agent framework runs on Python. Skip this and everything later breaks in mysterious ways.

1.1 Core Python

- Variables, types, control flow
- Functions, *args/**kwargs, decorators
- List & dict comprehensions
- Generator expressions
- Type hints (you'll need these for Pydantic later)

1.2 Object-Oriented Python

- Classes, __init__, instance vs class methods
- Inheritance, encapsulation, polymorphism
- Dataclasses
- Pydantic models — every agent framework uses them for tool schemas

1.3 Working with HTTP APIs

- The requests library
- HTTP verbs, headers, status codes
- Authentication (Bearer tokens, API keys)
- Rate limits, retries, exponential backoff with tenacity

1.4 Database Connectivity

- psycopg2 for raw PostgreSQL
- SQLAlchemy ORM basics
- Connection pooling and why it matters under load
- Raw SQL when the ORM gets in the way

1.5 FastAPI

- First /chat endpoint
- Pydantic request/response models
- Dependency injection
- Automatic OpenAPI docs
- Running with uvicorn

1.6 Async Programming

- asyncio fundamentals — event loop, coroutines
- async/await syntax
- asyncio.gather for parallel LLM calls
- asyncio.wait_for for timeout protection
- asyncio.create_task for fire-and-forget logging

END STATE

You can build a FastAPI endpoint that calls three different LLMs in parallel, times out the slow one, and logs the result without blocking the response.

02

The Mental Model of an LLM

Conceptual phase. Almost no code. Where the brain-in-a-windowless-room analogy lives, and where most "why is my agent broken" questions get answered six months later.

2.1 What an LLM actually is

- Trained on a fixed snapshot
- Knowledge cutoff dates and what they imply
- Probabilistic generation, not retrieval
- Why the same prompt gives different outputs

2.2 How an LLM thinks

- BPE tokenization — why "hello" is 1 token but "antidisestablishmentarianism" is 6
- Context windows — what fits, what gets silently truncated
- Sampling parameters: temperature, top-p, top-k — when to set what
- Transformer at 30,000 feet — attention preserves position, no math, no multi-head diagrams
- Why long context degrades ("lost in the middle")

2.3 Reasoning models vs base models

- The 2025 split: o1 / o3, Claude 3.7 thinking, Gemini 2.5 thinking, DeepSeek R1, Qwen QwQ
- What "thinking tokens" actually are and why they're billed
- When reasoning models are worth the latency and cost
- Reasoning effort knobs (low / medium / high) and how to budget them
- When a base model + good prompting beats a thinking model

2.4 Reading model evals & benchmarks

- The benchmarks worth knowing — MMLU, GSM8K, HumanEval, SWE-bench, GPQA, MMMU, BFCL (function calling)
- Why benchmarks lie — contamination, prompt sensitivity, eval gaming
- How to read a leaderboard skeptically (LMArena, Artificial Analysis, Vellum, Aider)
- Building your own micro-eval for the task you actually care about

2.5 Comparing the major models

- GPT family, Claude family, Gemini, Llama, Mistral, DeepSeek, Qwen
- Cost vs quality vs speed vs context-length tradeoffs
- When model choice matters vs when it really doesn't

END STATE

You can explain to a non-technical PM why ChatGPT made up a fact, and tell a hiring panel which model to pick for which job — backed by benchmarks, not vibes.

03 Prompt Engineering & API Access

The pivot from "ChatGPT user" to "engineer who controls LLMs."

3.1 UI vs API — the hinge moment

- Same prompt, same model, different output — why?
- System prompts you don't see
- Skills/tools the chat UI calls silently
- Why production work happens via API

3.2 Calling LLMs via API

- OpenAI SDK, Anthropic SDK
- Message format (system / user / assistant)
- Streaming responses
- Structured output (JSON mode, tool-call schemas, XML tags)

3.3 Prompt anatomy

- System prompt vs user turn vs assistant prefill
- Role and persona assignment
- Positive framing over negative constraints
- Markdown vs XML structure

3.4 Core techniques

- Zero-shot
- Few-shot with curated examples
- COSTAR framework (Context, Objective, Style, Tone, Audience, Response)
- Iterative refinement loop

3.5 Applied prompt patterns

- Extraction (entities, dates, relationships)
- Classification (intent, sentiment, routing)
- Transformation (summarize, translate, reformat)
- Generation (reports, SQL, code)
- Decomposition (break complex queries into sub-prompts)

3.6 Advanced reasoning techniques

- Chain of Thought — "think step by step"
- Self-Consistency — sample multiple paths, majority vote
- Self-Refine — generate, critique, refine loop
- Least-to-Most — decompose hard problems into ordered sub-problems
- Tree of Thought (research-flavoured; mention but don't drill)

3.7 Prompt management & cost in production

- Versioning prompts in code vs as managed resources
- A/B testing prompt variants
- AWS Bedrock Prompt Management for the lifecycle without code deploys
- Prompt caching — Anthropic `cache_control` and OpenAI's automatic cached input pricing (5–10× cost cuts on long system prompts)
- DSPy — programmatic prompt optimisation when you want the framework to tune your prompts for you (mention; depth is optional)

END STATE

You can take a flaky prompt that works "sometimes" and systematically make it reliable — and cut its cost in half with caching while you're at it.

04 RAG + Evaluation

The longest phase. RAG looks simple in tutorials and is brutal in production.

4.1 Why RAG exists

- LLMs can't see your private data
- The brain-in-a-windowless-room reaches its limit
- Use cases: internal docs, company policies, recent data

4.2 Embeddings

- What an embedding actually is (vector in N-dim space)
- Cosine similarity, dot product, Euclidean distance
- Embedding models — Titan Multimodal, SentenceTransformer, OpenAI ada/text-embedding-3, Cohere
- Choosing dimensions vs cost

4.3 Document ingestion pipeline

- Layout identification with Docling (headers, paragraphs, tables, code blocks, formulas)
- Serialization to structured objects
- Why PyMuPDF alone fails on complex PDFs

4.4 Chunking strategies

- Fixed-width chunking and why it breaks
- Semantic chunking by structure
- Overlap windows
- Parent-child chunking
- Late chunking — embed first, chunk later — preserves context across boundaries
- Chunk size vs retrieval quality tradeoff

4.5 Chunk enrichment

- PII detection and redaction
- NER for entities
- Key-phrase extraction
- Metadata for hybrid search

4.6 Vector databases

- Pinecone, Weaviate, pgvector
- Chroma for local dev
- S3 Vector Buckets, OpenSearch
- HNSW vs IVF indexes
- Decision matrix: managed (Pinecone) vs self-hosted (Weaviate, Qdrant) vs in-process (Chroma, FAISS) vs already-in-your-stack (pgvector)

4.7 Hybrid retrieval & next-gen retrievers

- Vector search + BM25 keyword
- Reranking with cross-encoders (Cohere Rerank, BGE)
- Metadata filtering
- Query expansion
- Late-interaction retrievers — ColBERT (text), ColPali (multimodal/PDF pages as images) — when they beat dense retrieval and what they cost

4.8 Graph-augmented RAG

- Neo4j basics
- Cypher query language
- When graph relationships beat pure vector search
- Multi-hop queries

4.9 RAG evaluation — the part most courses skip

- LLM-as-judge: RAG Triad — Faithfulness, Context Relevance, Answer Relevance
- Deterministic retrieval metrics: Precision@k, Recall@k, F1, Hit Rate@k, MRR, NDCG@k
- Tooling: Ragas (the de-facto eval framework), MLflow for run logging, LangSmith for tracing
- Golden datasets: Q&A pairs with expected chunks, regression testing on every code change

END STATE

You can build a RAG system, measure why it's wrong, and fix it with data instead of vibes.

05 Tools, MCP, and Single Agents

The brain gets hands and legs.

5.1 Function calling / tool use

- Tool schemas (JSON Schema, Pydantic)
- How the LLM decides which tool to call
- Parsing tool-call responses
- Handling tool errors gracefully

5.2 Tool design principles

- One tool, one job
- Clear docstrings — the LLM reads them
- Return structured data, not free text
- Fallbacks inside tools, not in the agent

5.3 MCP — Model Context Protocol

- What MCP is and why it exists (universal adapter for tools)
- MCP servers vs MCP clients
- Using existing MCP servers (filesystem, GitHub, Slack)
- Building your own MCP server
- stdio vs HTTP transports
- MCP is moving fast — bookmark modelcontextprotocol.io and re-read the spec every few months; the registry, auth model, and resource semantics are still evolving

5.4 The ReAct pattern

- Reasoning + Acting loop
- Thought → action → observation → thought
- Why "thinking" models exist
- When to force ReAct vs let the model decide

5.5 LangChain agents

- `create_agent` — model + tools + middleware + store
- `@tool(parse_docstring=True)` for auto schemas
- Parallel tool execution with `asyncio.gather`
- Structured outputs via Pydantic

5.6 Human in the loop

- `HumanInTheLoopMiddleware` for sensitive operations
- Checkpointers and `InMemorySaver`
- Resume flows after human approval
- When to pause (DB writes, payments, emails)

5.7 Tool security

- Retrieval Sanitiser — strip injection patterns from tool results
- Read-only DB enforcement
- Max retries per tool
- Timeouts on every external call

5.8 Computer use & app SDKs — agents with eyes and a mouse

- Anthropic Computer Use — agent takes screenshots and drives a desktop/browser
- OpenAI Operator / Apps SDK — agent runs inside ChatGPT or controls a browser tab
- Browser-automation agents (Playwright + LLM, browser-use, Stagehand)
- When this is the right tool vs API integration
- Sandboxing, audit trails, and "are you sure?" gates — these agents can do real damage

END STATE

You can build a single agent that searches the web, reads internal docs, queries a DB, and emails you a summary — and stops if it tries to do something dumb.

06 Memory & Context Engineering

The hardest conceptual phase. Easy to do badly, expensive when you do. Worth every hour of attention.

6.1 The context window as working memory

- Why agents "forget" mid-conversation
- Token budgeting per section
- The lost-in-the-middle problem
- Recency bias

6.2 Context structure — SYSTEM / CONTEXT / USER separation

- What goes where
- @dynamic_prompt patterns
- Structural separation as a security defence against prompt injection
- Token budgets per section (e.g. SYSTEM=instructions, CONTEXT=retrieved data, ~2000 tokens each)

6.3 Short-term memory — session history

- Sliding window of last N turns
- Message-pair preservation (don't split user from assistant)
- When to keep tool calls in history vs strip them

6.4 Semantic caching

- FAISS IndexFlatIP for sub-millisecond cosine search
- Similarity thresholds (0.97 high-stakes, 0.88 general Q&A)
- Cache HIT skips everything downstream
- Daemon-thread writes so cache never blocks response

6.5 Episodic memory

- LangChain's InMemoryStore
- LLM tags answers as EPISODIC: YES/NO so the model decides what's worth remembering
- Episodic hits enrich CONTEXT only — tools and LLM still run

6.6 Context compression

- Trigger threshold (>3000 tokens)
- Keep last 10 messages verbatim
- LLM summarises the rest into a single compressed entry
- When compression destroys information

6.7 Long-term memory

- User profiles, preferences, facts to persist
- Vector stores vs structured stores
- Managed memory layers — mem0 (open-source) and Zep (managed) — when to skip building this yourself
- When memory becomes a privacy problem (GDPR, right-to-be-forgotten flows)

END STATE

You can explain why your agent forgot what you said three turns ago, and fix it with the right memory layer instead of throwing more tokens at it.

07 Multi-Agent Orchestration

When one agent isn't enough.

7.1 When to go multi-agent (and when not to)

- Single-agent-with-tools beats multi-agent for ~80% of tasks
- Multi-agent earns its weight when steps need different prompts, tools, or specialised reasoning
- The Tableau→QuickSight conversion case as a worked example

7.3 Common patterns

- Supervisor + workers
- Sequential pipeline
- Parallel fan-out / fan-in
- Plan-and-execute
- Reflection loops

7.5 State management

- Typed state with Pydantic
- What to put in state vs context
- Checkpointers for resumability (MemorySaver, SqliteSaver, PostgresSaver)

7.7 Frameworks compared (briefly)

- LangGraph (most mature)
- CrewAI (simpler, opinionated)
- AutoGen (Microsoft)
- Pydantic AI (typed, FastAPI-flavoured ergonomics)
- OpenAI Swarm / its successor — minimal handoff-style orchestration
- Custom orchestration with raw asyncio
- Pick one and stick with it

7.2 LangGraph fundamentals

- Nodes, edges, state
- StateGraph and reducers
- Conditional edges and routing
- Cycles and termination conditions

7.4 Agent-as-tool — the lightweight alternative

- Wrap a sub-agent behind a normal @tool interface
- Parent agent calls it like any other function — no graph, no state plumbing
- When this beats LangGraph (clear hierarchy, no shared state, deterministic flow)
- Composing specialist agents (researcher, summariser, critic) without orchestration overhead

7.6 A2A — Agent-to-Agent Protocol

- Agent discovery and capability cards
- Cross-framework delegation
- When A2A beats just calling another function

7.8 Debugging multi-agent systems

- LangSmith tracing
- Why your agents are talking past each other
- Cycles that won't terminate
- Cost explosions

END STATE

You can design a multi-step agent workflow on a whiteboard, build it in LangGraph, and debug it when one node loops infinitely.

08

Guardrails & LLMOps

You know what to build. Now make it not embarrass you in production — measure failure, catch it before users do, and prove the agent is improving release-over-release.

8.1 Three-layer guardrail architecture

- Input Guardrails (gateway, <1ms, deterministic): prompt-injection regex, PII redaction, out-of-domain rejection, toxic filter — code-based, never LLM
- Output Guardrails (LLM-judge OK): faithfulness, contradiction check, medical/legal disclaimers when confidence < threshold, hard-fail to safe fallback
- Action Guardrails (inside tools, pure functions): max retries, max tool calls per request, query validation, read-only DB, top_k caps

8.2 AWS Bedrock Guardrails

- Contextual grounding
- Automated reasoning checks
- Harmful content filtering
- Topic blocking
- When managed guardrails are enough vs custom

8.3 LLMOps — observability

- LangSmith / LangFuse for traces
- Token cost dashboards
- Latency percentiles (p50, p95, p99)
- Failure rate by tool, by route, by model

8.4 LLMOps — evaluation in production

- Golden dataset regression tests in CI
- A/B testing prompt and model changes
- Feedback loops from user thumbs-up/down
- Drift detection on retrieval quality

END STATE

You can put a number on how often your agent fails, and ship it anyway with confidence.

09

Cloud Infrastructure & Deployment

The final mile. Minimum AWS to make everything earlier deployable, plus how to actually put an agent in production and keep costs sane.

9.1 Storage & data

- S3 — durable object storage, document lakes
- RDS PostgreSQL — managed relational DB for agent state
- DynamoDB — KV state for ingestion pipelines

9.2 Compute

- Lambda — serverless event-driven flows
- ECS Fargate — serverless containers for long-running agents
- ECR — container registry

9.3 Networking & access

- VPC, subnets, security groups (just enough not to break)
- IAM roles and policies
- API Gateway for exposing endpoints

9.4 AI-specific services (and other clouds)

- AWS Bedrock — managed foundation models
- AWS AgentCore — production agent infrastructure
- Bedrock embeddings
- Equivalents on other clouds: GCP Vertex AI (Model Garden, Agent Builder) and Azure AI Foundry (model catalog, prompt flow) — same primitives, different SKUs

9.5 Deployment & realtime delivery

- Dockerizing FastAPI agents
- ECS Fargate task definitions
- API Gateway + ALB routing
- Secrets management with AWS Secrets Manager
- Environment promotion (dev → staging → prod)
- Streaming responses to chat UIs — SSE for one-way token streaming, WebSockets when you also need client → server messages mid-stream

9.6 Cost & capacity control

- Semantic cache HIT rate as a KPI
- Model routing — cheap model for simple queries, expensive for complex
- Prompt compression
- Max-tokens caps
- Load testing with locust or k6 — agents fall over under concurrency long before the LLM does; rate-limit at the gateway, not the model

END STATE

You can take any system you built in earlier phases, dockerize it, deploy to ECS Fargate behind API Gateway, manage secrets, stream tokens to a chat UI, load-test it, and watch the cost dashboard move only when it should.

Capstone projects

Three production-grade systems, built as you go. These are your portfolio — the proof you can ship, not just follow tutorials.

CAPSTONE 1

Distributed Document Ingestion + RAG Pipeline

Built during Phase 4 · Weeks 10-12

Domain: Unstructured document Q&A (legal, pharma, technical docs)

WHAT YOU BUILD

- PDF ingestion: Docling layout detection → semantic chunking → PII redaction → entity extraction → embeddings → Pinecone + Neo4j
- Distributed async workers on ECS Fargate processing thousands of PDFs concurrently
- DynamoDB state tracking per document (queued / processing / done / failed)
- Hybrid retrieval (vector + BM25 + graph) with reranking
- Evaluation harness with golden dataset, Precision@k / Recall@k / RAG Triad
- FastAPI Q&A endpoint with citation-backed answers

STACK

Docling

Pinecone

Neo4j

ECS Fargate

DynamoDB

S3

Bedrock embeddings

LangSmith

Proves: You can build production RAG, not a Streamlit demo.

CAPSTONE 2

Multi-Agent Natural Language → SQL on E-commerce Data

Built during Phase 7 • Weeks 21–22

Domain: E-commerce analytics for non-technical users

WHAT YOU BUILD

- Multi-agent: Planner → SQL Writer → Validator → Executor → Explainer
- Schema-aware context injection per query (only relevant tables sent to writer)
- LangGraph orchestration with conditional routing and retry loops
- Read-only DB enforcement, query timeout, max-row caps
- Streamlit frontend, FastAPI backend, RDS PostgreSQL with realistic data
- Benchmarked on a golden NLQ test set, target 85%+ accuracy

STACK

LangChain

LangGraph

LangSmith

AgentCore

RDS PostgreSQL

FastAPI

Streamlit

Bedrock

Proves: You can orchestrate multiple specialised agents safely against real production data.

CAPSTONE 3

Clinical Trials Knowledge Base

Built during Phases 8–9 • Weeks 23–26

Domain: Life sciences AI (substitute legal, finance, or your industry)

WHAT YOU BUILD

- Real ClinicalTrials.gov dataset ingestion (or your domain equivalent)
- Hybrid knowledge layer: Pinecone for unstructured PDFs + Neo4j for trial-drug-condition relationships
- Multi-hop relationship queries ("what other trials used drug X for condition Y?")
- Full three-layer guardrails — disclaimer auto-injection, contradiction checks, action limits
- Evidence-backed answers — every claim cites the source chunk
- Deployed on AWS with monitoring, regression tests in CI, semantic cache, cost dashboard

STACK

LangChain

LangGraph

Neo4j + Cypher

Pinecone

Bedrock + AgentCore + Lambda

S3

LangSmith

MLflow

Proves: You can ship an agent into a regulated domain without it killing anyone (or your career).

Deliberately out of scope

What this roadmap leaves out on purpose — and where to pick it up later, once you've shipped your first agent.

Fine-tuning foundation models

RAG, prompting, and tool use solve 95% of business problems faster, cheaper, and with no infra overhead. Fine-tuning earns its weight only when you have a narrow domain, lots of clean labelled data, and prompting has hit a wall — which almost never happens before you've shipped your first agent. Learn it after this roadmap, not during.

Where to start later: Start with LoRA + a 7B open model (Llama, Mistral, Qwen) on a single A10/L4 once you have a real motivating use case.

Voice agents

A whole sub-discipline — STT, TTS, turn-taking, latency budgets, barge-in. Worth its own track, not a side note. You can graft it on top of any agent you build in this roadmap.

Where to start later: OpenAI Realtime API, Deepgram + ElevenLabs + LiveKit, or pipecat — pick after you've shipped one text agent.

ML fundamentals (gradient descent, backprop, transformers from scratch)

Lovely to know. Not required to be an excellent agent engineer in 2026. The Karpathy series is there when you're curious — don't let it block you from shipping.

Where to start later: Andrej Karpathy's "Neural Networks: Zero to Hero" + the "Let's build GPT" video, on weekends.

Frontend frameworks (Next.js, React, Tailwind)

You need enough to ship a Streamlit or basic chat UI for capstones. Beyond that, partner with a frontend engineer or a design system. Don't get lost in framework wars.

Where to start later: Streamlit for internal tools, Vercel AI SDK + Next.js when you need a real product UI.

After the roadmap

You've built three systems. Here's how to turn them into a job.

PORTFOLIO

Three repos, three READMEs, one demo video each

The capstones are your portfolio. For each one: a clean GitHub repo with a README that explains the problem, the architecture, the trade-offs, and the eval numbers; a 90-second Loom walking through it; one screenshot of the trace UI showing it actually working.

LINKEDIN

Headline that says what you can ship

Don't write "AI Engineer" in your headline — write "AI Engineer · production RAG, multi-agent systems, AWS Bedrock + LangGraph · shipping in regulated domains." Specific gets interviews. Generic gets ignored.

60-SECOND PITCH

What to say in the first interview round

"I spent six months building three production-grade AI systems end-to-end: a distributed RAG pipeline that ingests thousands of PDFs, a multi-agent NL→SQL system with read-only enforcement, and a clinical-trials knowledge base with three-layer guardrails. I can show you the traces, the eval numbers, and the cost dashboard for any of them." That's the whole pitch. Numbers and artefacts beat adjectives.

KEEP LEARNING

What to read once you're shipping

Anthropic's "Building effective agents" essay, the Latent Space podcast, the LangChain blog, Eugene Yan's writing on production ML, and the original papers when something keeps confusing you (Self-RAG, RAG-as-judge, ReAct). Skim, don't drown.

The 2026 Agentic AI Engineer Roadmap · Balaji Chippada
Free & open at balajichippada.com/roadmap · YouTube @balajichippada